

Nearest Neighbors Methods

Intuition and Background

Agha Ali Raza



Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell University, Lecture 2, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- **Nearest Neighbor Methods**, Victor Lavrenko, Assistant Professor at the University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ
- **Wiki K-Nearest Neighbors**: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- **Effects of Distance Measure Choice on KNN Classifier Performance - A Review**, V. B. Surya Prasath et al., <https://arxiv.org/pdf/1708.04321.pdf>
- **A Comparative Analysis of Similarity Measures to find Coherent Documents**, Mausumi Goswami et al. <http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf>
- **A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data**, Ali Seyed Shirkhorshidi et al., <https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable>

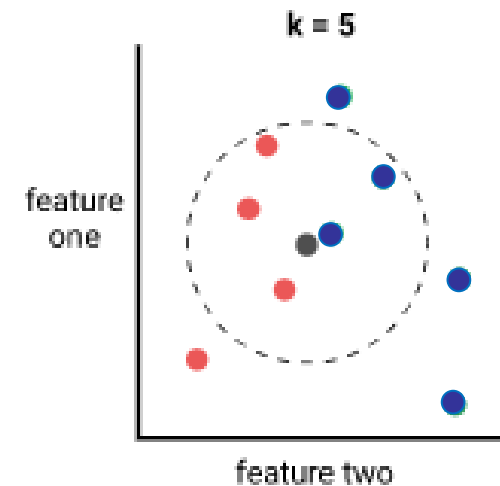
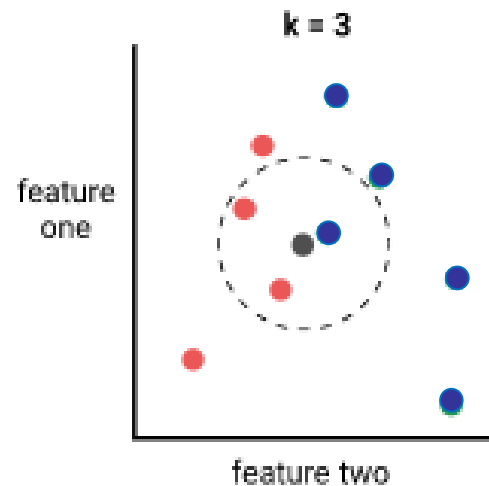
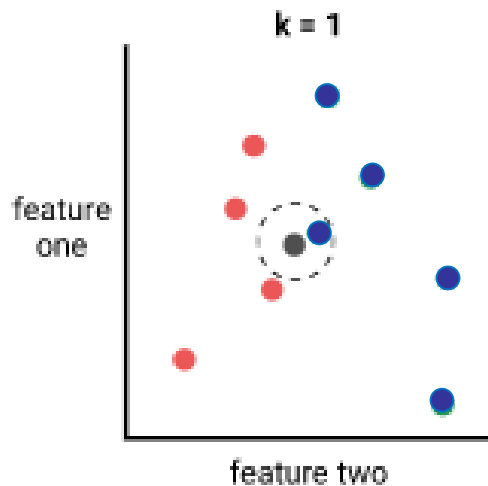
The K Nearest Neighbors Algorithm

“A **data point** is known by the **company** it keeps”

(Aesop – *the data scientist*)

“A **data point** is known by the most common **company** it keeps”

(Aesop – *the improving data scientist*)

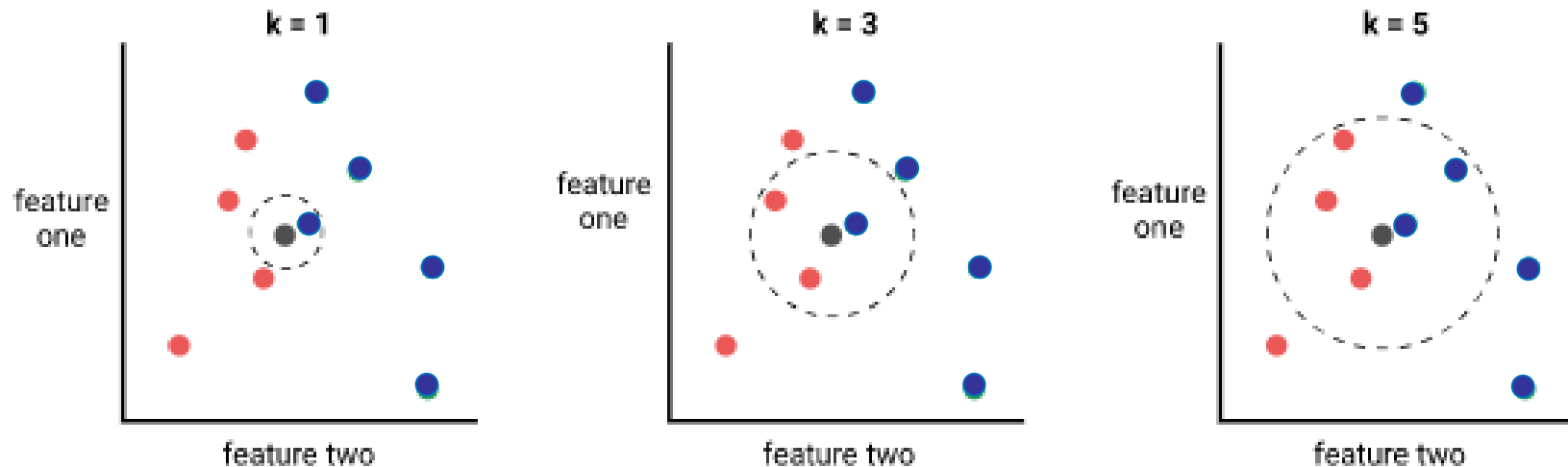


The K Nearest Neighbors Algorithm

Basic idea: Similar Inputs have similar outputs

Classification rule:

For a test input x , assign the most common label amongst its k most similar (nearest) training inputs



Formal Definition

Assuming x to be our test point, let's denote the set of the k nearest neighbors of x as S_x

Formally, S_x is defined as

$$\begin{aligned} S_x &\subseteq D \text{ s.t. } |S_x| = k \\ &\text{and} \\ \forall (x', y') \in D \setminus S_x, \\ \text{dist}(x, x') &\geq \max_{(x'', y'') \in S_x} \text{dist}(x, x''), \end{aligned}$$

That is, every point that is in D but not in S_x is at least as far away from x as the furthest point in S_x .

We define the classifier $h()$ as a function returning the most common label in S_x :

$$h(x) = \text{mode}(\{y'' : (x'', y'') \in S_x\}),$$

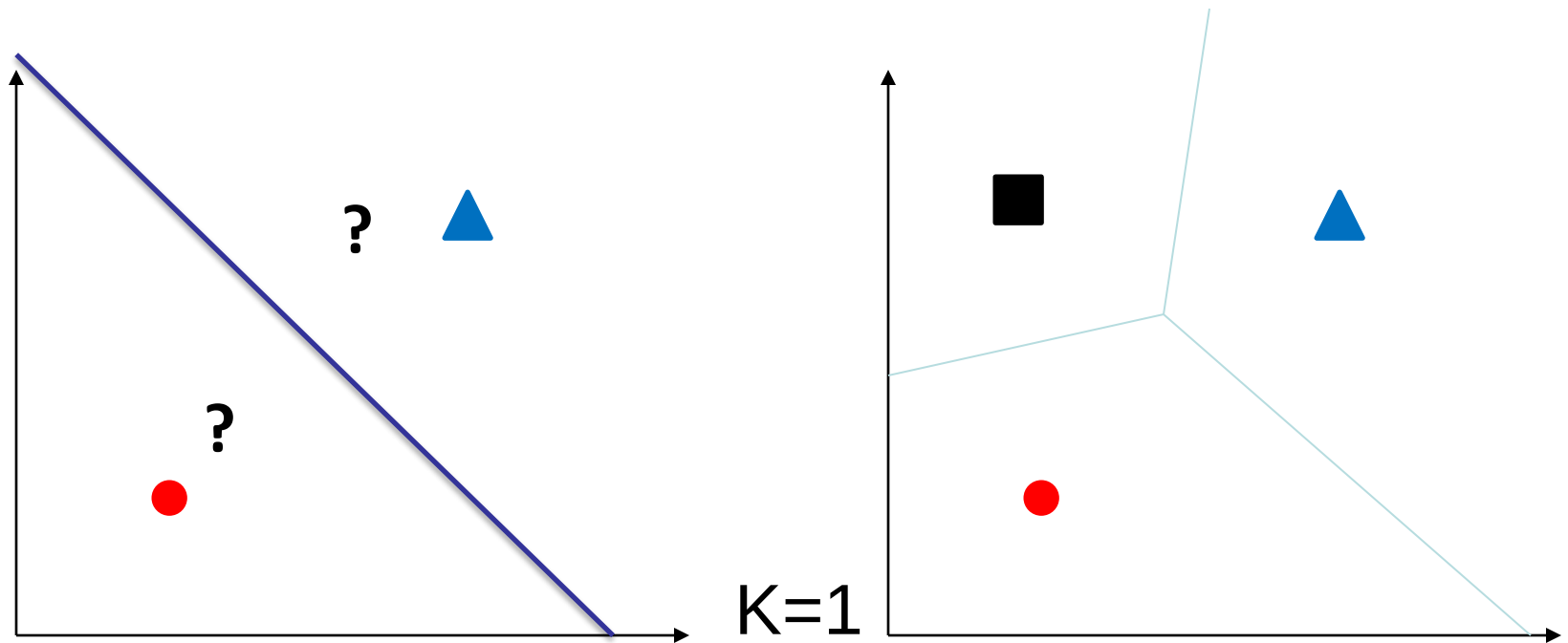
where $\text{mode}(\cdot)$ means to select the label of the highest occurrence.

So, what do we do if there is a draw?

- Keep k odd (for binary classification) or return the result of $k-1$ NN with a smaller k

KNN Decision Boundary

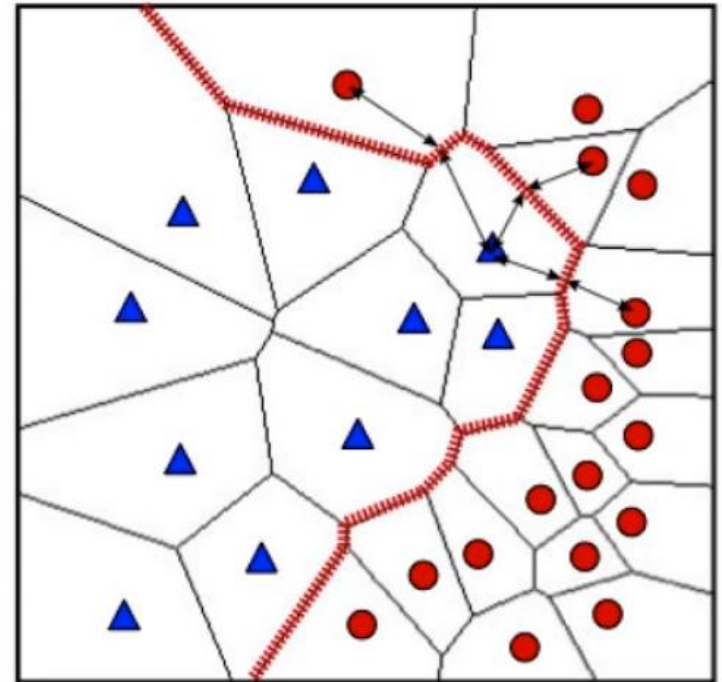
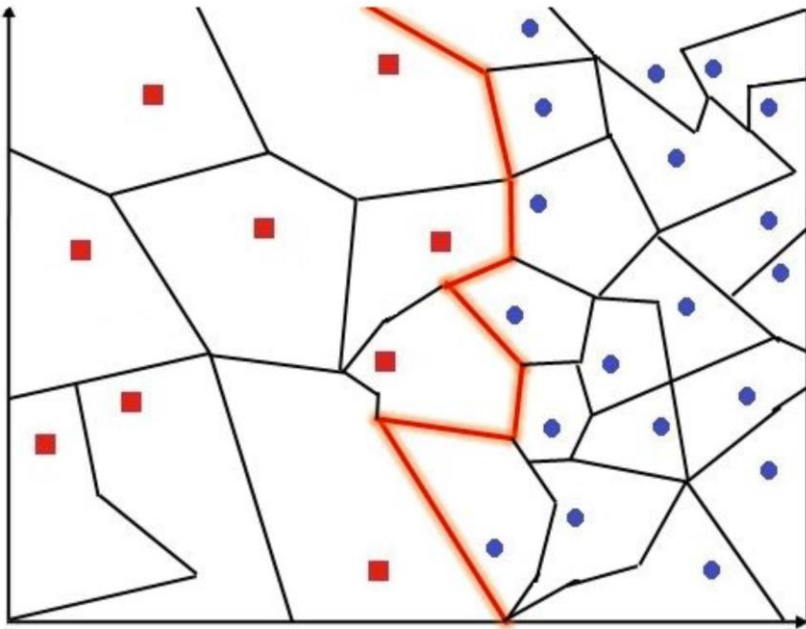
Voronoi Tessellation and KNN decision boundaries



KNN Decision Boundary

Voronoi Tessellation and KNN decision boundaries

$K=1$



Properties of KNN – Non-parametric

The KNN Algorithm is a **supervised, non-parametric** algorithm

- It does not make any assumptions about the underlying distribution nor tries to estimate it

Properties of KNN – Non-parametric

- **Parametric models** summarize data with a fixed set of parameters (independent of the number of training examples).
 - No matter how much data you throw at a parametric model, it won't change its mind about how many parameters it needs.
- **Non-parametric models** make no assumptions about the probability distribution or number of parameters when modeling the data.
 - Nonparametric methods are good when you have a lot of data and no prior knowledge, and when you don't want to worry too much about choosing just the right features.
 - **Non-parametric does not mean that they have no parameters!** On the contrary, non-parametric models (can) become more and more complex with an increasing amount of data.

Properties of KNN – Non-parametric

- **Differences:**

- In a parametric model, we have a finite number of parameters, and in nonparametric models, the number of parameters is (potentially) infinite.
- In statistics, the term parametric is also associated with a specified probability distribution that you “assume” our data follows, and this distribution comes with the finite number of parameters (for example, the mean and standard deviation of a normal distribution)
 - We don't make/have these assumptions in non-parametric models. So, in intuitive terms, we can think of a non-parametric model as a “distribution” or (quasi) assumption-free model.
- Still, the distinction is a bit ambiguous at best

Notes: Detailed Differences between Parametric and Non-Parametric Models

Parametric models assume some finite set of parameters for the model, while non-parametric models do not make any assumptions about the data distribution and have an infinite number of parameters. Here are some key differences:

Number of Parameters:

1. *Parametric Models:* These models have a fixed number of parameters. For example, in a linear regression model, the parameters are the slope and intercept.
2. *Non-parametric Models:* These models do not have a fixed number of parameters. The number of parameters grows with the amount of training data. For example, in a k-nearest neighbors (KNN) model, the "parameters" are essentially the entire training dataset.

Notes: Detailed Differences between Parametric and Non-Parametric Models

Flexibility:

1. *Parametric Models*: Less flexible as they make strong assumptions about the data distribution.
2. *Non-parametric Models*: More flexible as they make fewer assumptions about the data distribution.

Computational Complexity:

1. *Parametric Models*: Generally, less computationally intensive as they require estimating only a fixed number of parameters.
2. *Non-parametric Models*: Usually, more computationally intensive as they involve a larger number of parameters and often require computation over the entire dataset.

Detailed Differences between Parametric and Non-Parametric Models

Risk of Overfitting:

1. *Parametric Models*: Higher risk of underfitting as they might not capture the underlying complexity of the data.
2. *Non-parametric Models*: Higher risk of overfitting as they might capture too much noise in the data.

Examples:

- *Parametric Models*: Linear Regression, Logistic Regression, Naive Bayes, etc.
- *Non-parametric Models*: Decision Trees with unbounded height, k-Nearest Neighbors, Support Vector Machines, etc.

Properties of KNN – Hyperparameter K

Parameters and hyperparameters are two types of values that a model uses to make predictions, but they serve different purposes and are learned in different ways:

Parameters:

- These are the parts of the model that are learned from the training data. For example, the weights in a linear regression model are parameters.
- Parameters are learned directly from the training data during the training process. The model uses the training data to adjust the parameters to minimize the prediction error.

Hyperparameters:

- These are the settings or configurations that need to be specified prior to training the model. They are not learned from the data but are essential for the learning process.
- For example, the learning rate in gradient descent, the depth of a decision tree, or the number of clusters in k-means clustering are all hyperparameters.
- The values of hyperparameters are usually set before training the model and remain constant during the training process. They may be adjusted between runs of training to optimize model performance.

Properties of KNN

- Used for **classification** and **regression**
 - **Classification:** Choose the most frequent class label amongst k-nearest neighbors
 - **Regression:** Take an average over the output values of the k-nearest neighbors and assign to the test point – may be weighted e.g. $w = \frac{1}{d}$ (d : distance from x)

Properties of KNN

An **Instance-based** learning algorithm

- Instead of performing explicit generalization, form hypotheses by comparing new problem instances with training instances
- (+) Can easily adapt to unseen data
- (-) Complexity of prediction is a function of n (size of training data)

A **lazy** learning algorithm

- Delay computations on training data until a query is made, as opposed to **eager learning**
- (+) Good for continuously updated training data like recommender systems
- (-) Slower to evaluate and need to store the whole training data

Nearest Neighbors Methods

Distance/Similarity

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell University, Lecture 2, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- **Nearest Neighbor Methods**, Victor Lavrenko, Assistant Professor at the University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ
- **Wiki K-Nearest Neighbors**: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- **Effects of Distance Measure Choice on KNN Classifier Performance - A Review**, V. B. Surya Prasath et al., <https://arxiv.org/pdf/1708.04321.pdf>
- **A Comparative Analysis of Similarity Measures to find Coherent Documents**, Mausumi Goswami et al. <http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf>
- **A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data**, Ali Seyed Shirkhorshidi et al., <https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable>

Similarity/Distance Measures

- If scaled between 0 and 1, then $sim = 1 - distance$
- Lots of choices, depends on the problem
- The **Minkowski distance** is a generalized metric form of Euclidean, Manhattan and Chebyshev distances
- The Minkowski distance between two n-dimensional vectors $P = \langle p_1, p_2, \dots, p_n \rangle$ and $Q = \langle q_1, q_2, \dots, q_n \rangle$, it is defined as:

$$d_{minkowski}(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^a \right)^{1/a}, a \geq 1$$

- $a = 1$, is the Manhattan distance
- $a = 2$, is the Euclidean distance
- $a \rightarrow \infty$, is the Chebyshev distance

Constraints on Distance Metrics

The distance function between vectors p and q is a function $d(p, q)$ that defines the distance between both vectors is considered as a metric if it satisfy a certain number of properties:

1. Non-negativity: The distance between p and q is always a value greater than or equal to zero

$$d(p, q) \geq 0$$

2. Identity of indiscernible vectors: The distance between p and q is equal to zero if and only if p is equal to q

$$d(p, q) = 0 \text{ iff } p = q$$

3. Symmetry: The distance between p and q is equal to the distance between q and p .

$$d(p, q) = d(q, p)$$

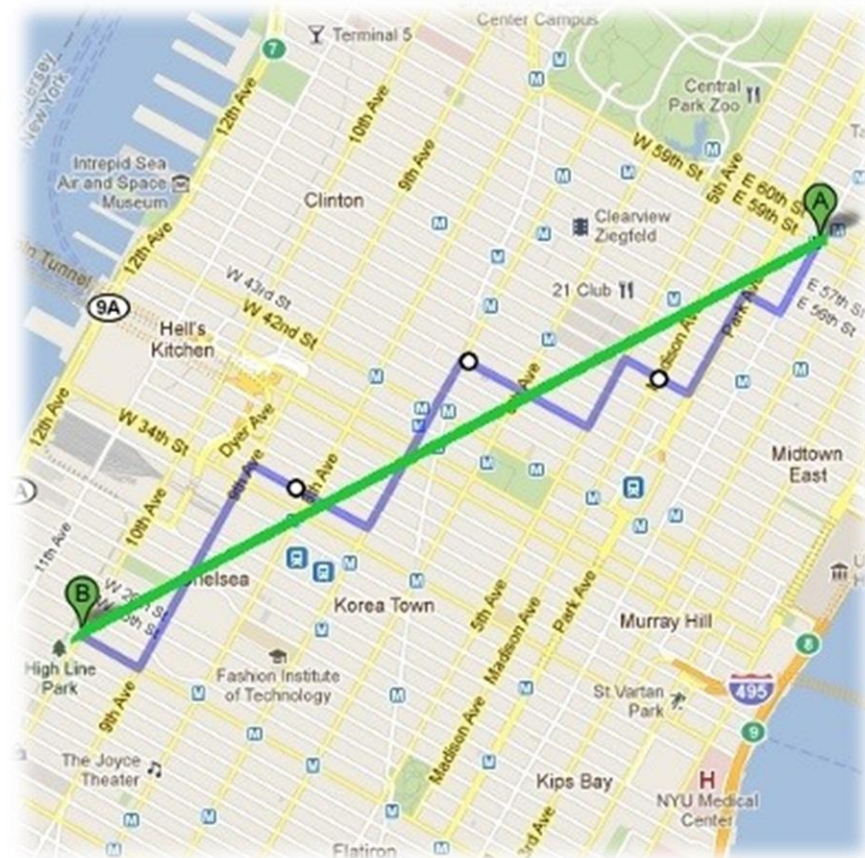
4. Triangle inequality: Given a third point r , the distance between p and q is always less than or equal to the sum of the distance between p and r and the distance between r and q

$$d(p, q) \leq d(p, r) + d(r, q)$$

Manhattan Distance

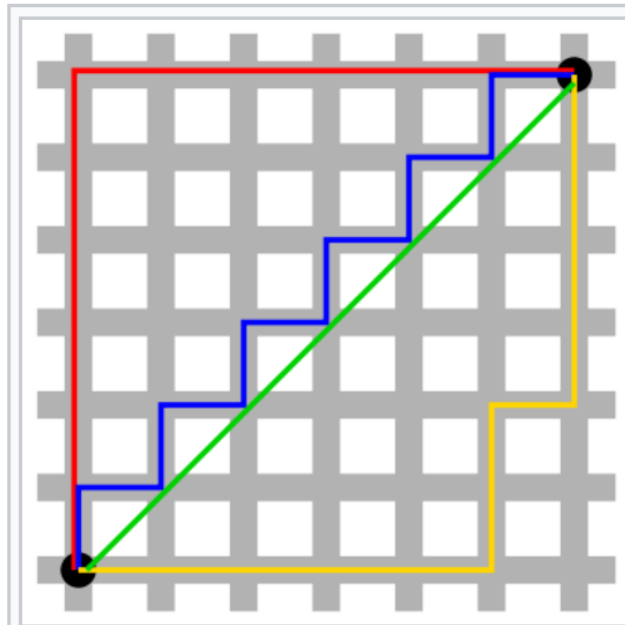
$$d_{Man}(p, q) = d(q, p) = |p_1 - q_1| + |p_2 - q_2| + \dots + |p_n - q_n|$$
$$d(p, q) = d(q, p) = \sum_{i=1}^n |p_i - q_i|$$

- The distance between two points is the sum of the absolute differences of their Cartesian coordinates.
 - It is the total sum of the difference between the x-coordinates and y-coordinates.
- Also known as Manhattan length, rectilinear distance, L1 distance or L1 norm, city block distance, snake distance, taxi-cab metric, or city block distance
- Works well for high dimensional data.
 - It does not amplify differences among features of the two vectors and as a result does not ignore the effects of any feature dimensions
 - Higher values of α amplify differences and ignore features with smaller differences



Euclidean vs Manhattan Distance

https://en.wikipedia.org/wiki/Taxicab_geometry



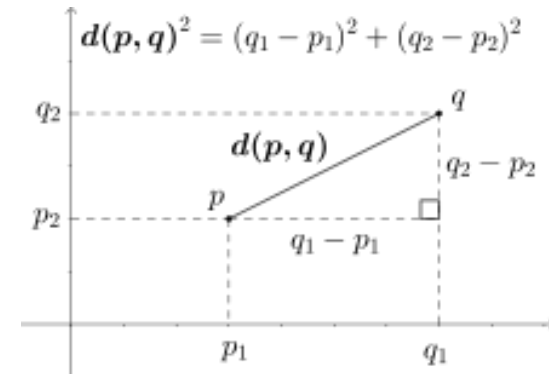
Taxicab geometry versus
Euclidean distance: In taxicab
geometry, the red, yellow, and blue
paths all have the same shortest
path length of 12. In Euclidean
geometry, the green line has length
 $6\sqrt{2} \approx 8.49$, and is the unique
shortest path.



Euclidean Distance

$$d(p, q) = d(q, p) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2}$$

$$d(p, q) = d(q, p) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$



- Good choice for numeric attributes
- When data is dense or continuous, this is a good proximity measure
- Downside: Sensitive to extreme deviations in attributes (as it squares differences)
 - The variables which have the largest value greatly influence the result
 - Does not work well for situations where features on different scales are mixed (e.g., #bedrooms (1-5) and area (200 – 5,000 sq feet) of a house)
 - Solution: feature normalization (min-max scaling)

Chebyshev Distance

Effect of Different Distance Measures in Result of Cluster Analysis, Sujan Dahal

$$d_{Cheb}(p, q) = \lim_{a \rightarrow \infty} \left(\sum_{i=1}^n |p_i - q_i|^a \right)^{1/a} = \max_i |p_i - q_i|$$

How?

Assume, $p = \langle 2, 3, \dots, 9 \rangle$, $q = \langle 4, 6, \dots, 10 \rangle$

$$d_{Cheb}(p, q) = \lim_{a \rightarrow \infty} (|2 - 4|^a + |3 - 6|^a + \dots + |9 - 10|^a)^{\frac{1}{a}}$$

$$d_{Cheb}(p, q) = \lim_{a \rightarrow \infty} (2^a + 3^a + \dots + 1^a)^{\frac{1}{a}}$$

Suppose, $a = 2$

$$d(p, q) = (4 + 9 + \dots + 1)^{\frac{1}{2}}$$

Suppose, $a = 3$

$$d(p, q) = (8 + 27 + \dots + 1)^{\frac{1}{3}}$$

Suppose, $a = 10$

$$d(p, q) = (1,024 + 59,049 + \dots + 1)^{\frac{1}{10}}$$

Now, $a \rightarrow \infty$

$$d_{Cheb}(p, q) = \lim_{a \rightarrow \infty} \left(\sum_{i=1}^n |p_i - q_i|^a \right)^{\frac{1}{a}} \rightarrow \max_i |p_i - q_i|^a)^{\frac{1}{a}}$$

$$d_{Cheb}(p, q) = \lim_{a \rightarrow \infty} (\max_i |p_i - q_i|^a)^{\frac{1}{a}}$$

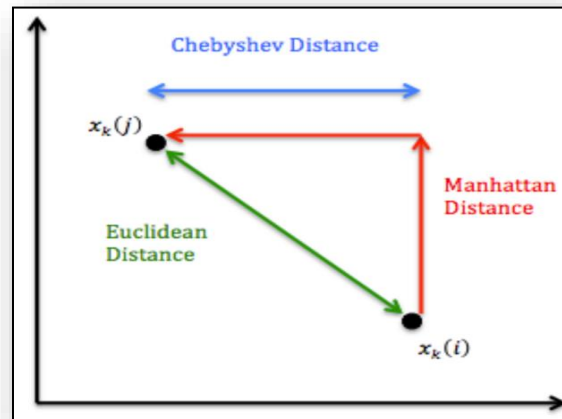
$$d_{Cheb}(p, q) = \max_i |p_i - q_i|$$

Chebyshev Distance

Effect of Different Distance Measures in Result of Cluster Analysis, Sujan Dahal

$$d_{cheb}(p, q) = \max_i |p_i - q_i|$$

- For Chebyshev distance, the distance between two vectors is the greatest of their differences along any coordinate dimension
- When two objects are to be defined as “different”, if they are different in any one dimension
- Also called chessboard distance, maximum metric, or L_∞ metric



Nearest Neighbors Methods

The KNN Algorithm

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell University, Lecture 2, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- **Nearest Neighbor Methods**, Victor Lavrenko, Assistant Professor at the University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ
- **Wiki K-Nearest Neighbors**: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- **Effects of Distance Measure Choice on KNN Classifier Performance - A Review**, V. B. Surya Prasath et al., <https://arxiv.org/pdf/1708.04321.pdf>
- **A Comparative Analysis of Similarity Measures to find Coherent Documents**, Mausumi Goswami et al. <http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf>
- **A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data**, Ali Seyed Shirkhorshidi et al., <https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable>

The KNN Algorithm

Input: Training samples $D = \{(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_n, y_n)\}$, Test sample $d = (\vec{x}, y)$, k . Assume $\vec{x}$ to be an m -dimensional vector.

Output: Class label of test sample d

1. Compute the distance between d and every sample in D
2. Choose the K samples in D that are nearest to d ; denote the set by $S_d \in D$
3. Assign d the label y_i of the majority class in S_d

Note:

All action takes place in the test phase, the training phase is essentially to clean, normalize and store the data

KNN Classification and Regression

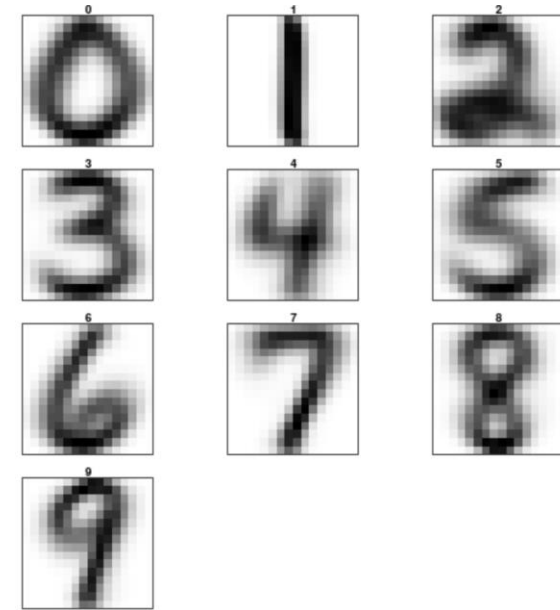
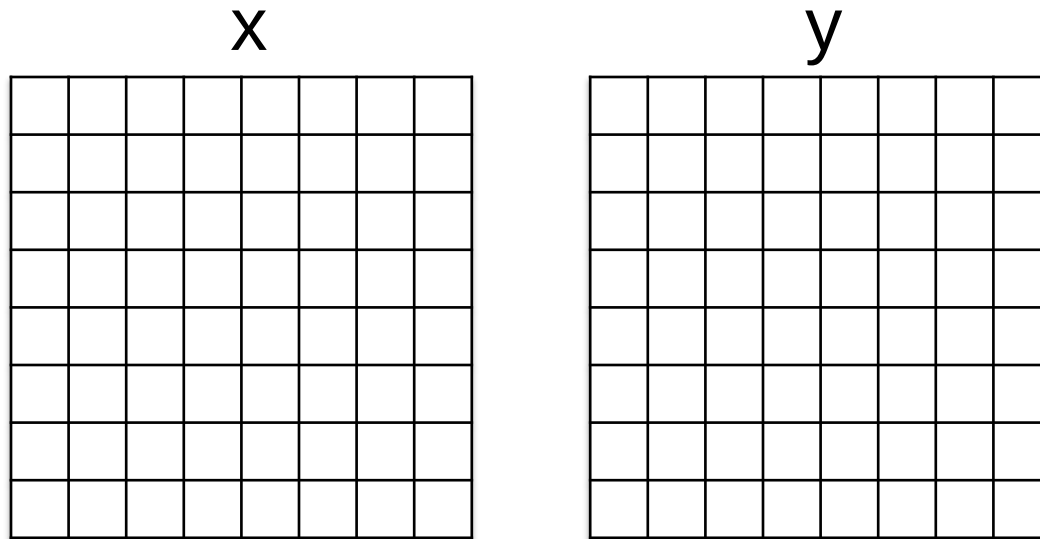
#	Height (inches)	Weight (kgs)	B.P. Sys	B.P. Dia	Heart disease	Cholesterol Level	Euclidean Distance
1	62	70	120	80	No	150	52.59
2	72	90	110	70	No	160	47.81
3	74	80	130	70	No	130	43.75
4	65	120	150	90	Yes	200	7.14
5	67	100	140	85	Yes	190	16.61
6	64	110	130	90	No	130	15.94
7	69	150	170	100	Yes	250	44.26
8	66	115	145	90			

K=3

$$\frac{200 + 190 + 130}{3} = 173.33$$

Example: Handwritten digit recognition

- 16x16 bitmaps
- 8-bit grayscale
- Euclidean distances over raw pixels



- $$D(x, y) = \sqrt{\sum_{i=0}^{255} (x_i - y_i)^2}$$

Accuracy:

- 7-NN ~ 95.2%
- SVM ~ 95.8%
- Humans ~ 97.5%

http://rstudio-pubs-static.s3.amazonaws.com/6287_c079c40df6864b34808fa7ecb71d0f36.html,

Victor Lavrenko https://www.youtube.com/watch?v=ZD_tfNpKzHY&list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ&index=6

Complexity of KNN

Input: Training samples $D = \{(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_n, y_n)\}$, Test sample $d = (\vec{x}, y)$, k . Assume $\vec{x}$ to be an m -dimensional vector.

Output: Class label of test sample d

1. Compute the distance between d and every sample in D

n samples, each is m -dimensional $\Rightarrow O(mn)$

2. Choose the K samples in D that are nearest to d ; denote the set by $S_d \in D$

- Either naively do K passes of all samples costing $O(n)$ each time for $O(nk)$
- Or use the *quickselect* algorithm (median of medians) to find the k th smallest distance in $O(n)$ and then return all distances no larger than the k th smallest distance. This will accumulate to $O(n)$

3. Assign d the label y_i of the majority class in S_d

This is $O(k)$.

Time complexity: $O(mn + n + k) = O(mn)$, assuming k to be a constant.

Space complexity: $O(mn)$, to store the n , m -dimensional training data samples.

Tuning the Hyperparameter K

What happens if we use the training set itself as the test dataset, instead of a validation set? Which k wins?

- $K=1$, as there is always a nearest instance with the correct label:
The instance itself!

What happens if we use $K=n$?

- KNN will always return the majority class in the dataset

Choosing the value of K – the theory

k=1:

- High variance
- Small changes in the dataset will lead to big changes in classification
- Overfitting
 - Is too specific and not well-generalized
 - It tends to be sensitive to noise
 - The model accomplishes a high accuracy on train set but will be a poor predictor on new, previously unseen data points

k= very large (e.g., 100):

- The model is too generalized and not a good predictor on both train and test sets.
- High bias
- Underfitting

k=n:

- The majority class in the dataset wins for every prediction
- High bias

Tuning the hyperparameter K – the Method

1. Divide your training data into training and validation sets.
 2. Do multiple iterations of m-fold cross-validation, each time with a different value of k, starting from k=1
 3. Keep iterating until the k with the best classification accuracy (minimal loss) is found
-
- What happens if we use the training set itself, instead of a validation set? Which k wins?
 - K=1, as there is always a nearest instance with the correct label, the instance itself

KNN – The good, the bad and the ugly

KNN is a simple algorithm but is highly effective for solving various real life classification problems. Especially when the datasets are large and continuously growing.

- We can show that as $n \rightarrow \infty$, the 1-NN classifier is only a factor 2 worse than the best possible classifier (the Bayes Optimal Classifier).

Challenges:

1. How to find the optimum value of K?
2. How to find the right distance function?

Problems:

1. High computational time cost for each prediction.
2. High memory requirement as we need to keep all training samples.
3. The curse of dimensionality.



K Nearest Neighbors

Error as $n \rightarrow \infty$

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 2, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- **Wiki K-Nearest Neighbors**: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- **Effects of Distance Measure Choice on KNN Classifier Performance - A Review**, V. B. Surya Prasath et al., <https://arxiv.org/pdf/1708.04321.pdf>
- **A Comparative Analysis of Similarity Measures to find Coherent Documents**, Mausumi Goswami et al. <http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf>
- **A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data**, Ali Seyed Shirkhorshidi et al., <https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable>
- Cover, Thomas, and, Hart, Peter. Nearest neighbor pattern classification[J]. Information Theory, IEEE Transactions on, 1967, 13(1): 21-27

Bayes Error https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html

Assume that we know $P(y|x)$, so we can simply predict the correct label y^* as:

$$y^* = h_{opt}(x) = \operatorname{argmax}_y P(y|x)$$

Although the Bayes Classifier is optimal, but it is not perfect and can still make mistakes. E.g. It will predict incorrectly when a test point does not have the most likely label.

So, if $P(y^*|x)$ is the probability of correct classification then the probability of incorrect classification, the *Bayes Error*, is given as:

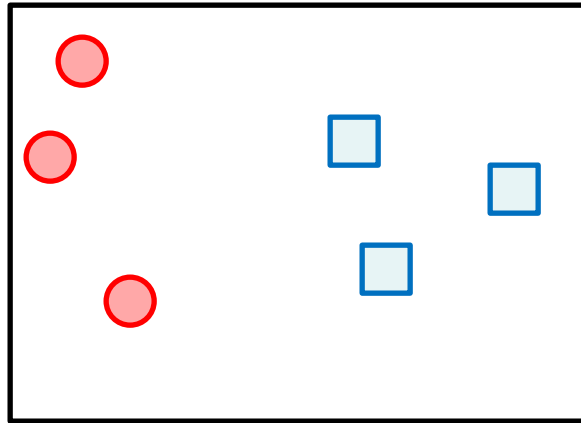
$$\epsilon_{Bayes} = 1 - P(y^*|x)$$

1-NN Error as $n \rightarrow \infty$ (Cover and Hart 1967, Weinberger Lec 2)

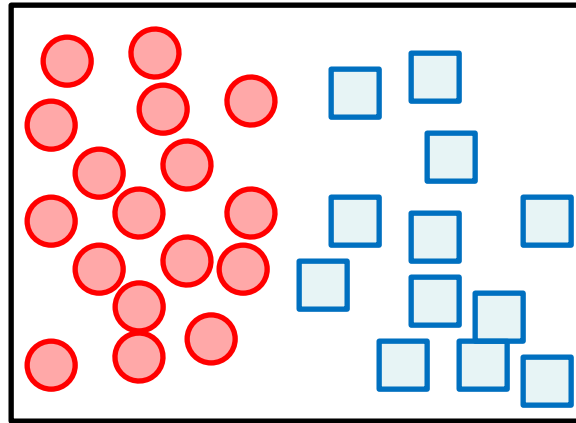
Let x_{NN} be the nearest neighbor of our test point x_t

As $n \rightarrow \infty$, $\text{dist}(x_{NN}, x_t) \rightarrow 0$

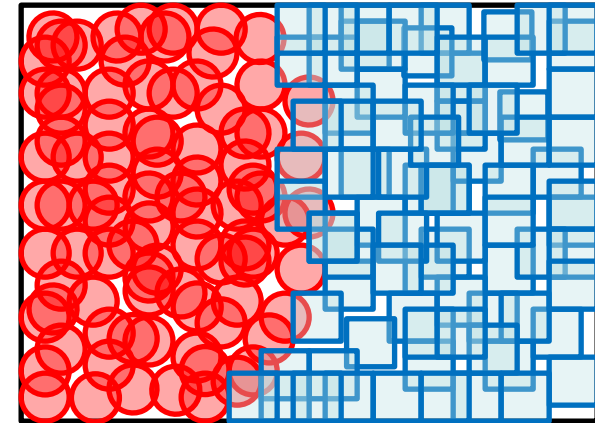
- i.e. $x_{NN} \rightarrow x_t$
- 1-NN returns the label of x_{NN}



Small n



Large n



$n \rightarrow \infty$

What is the probability that this is not the correct label of x_t ?

- As $x_{NN} \rightarrow x_t$, the probability of misclassification is the same as the probability of x_{NN} and x_t having different labels

1-NN Error as $n \rightarrow \infty$ (Cover and Hart 1967, Weinberger Lec 2)

There are two ways this could happen.

1. y^* was the correct label of x_t but the nearest neighbor x_{NN} did not have that label
2. y^* was not the correct label of x_t but the nearest neighbor x_{NN} had that label

Let's define some probabilities:

- The probability that x_{NN} had the correct label y^*
 - $P(y^*|x_{NN})$
- The probability that x_{NN} did not have the correct label y^*
 - $1 - P(y^*|x_{NN})$
- The probability that y^* was the correct label of x_t
 - $P(y^*|x_t)$
- The probability that y^* was not the correct label of x_t
 - $1 - P(y^*|x_t)$

Therefore:

1. The probability that y^* was the correct label of x_t but the nearest neighbor x_{NN} did not have that label:

$$P(y^*|x_t)(1 - P(y^*|x_{NN}))$$

2. The probability that y^* was not the correct label of x_t but the nearest neighbor x_{NN} had that label?

$$P(y^*|x_{NN})(1 - P(y^*|x_t))$$

1-NN Error as $n \rightarrow \infty$ (Cover and Hart 1967, Weinberger Lec 2)

So, the total probability of misclassification is:

$$\epsilon_{NN} = P(y^*|x_t)(1 - P(y^*|x_{NN})) + P(y^*|x_{NN})(1 - P(y^*|x_t))$$

As $P(y^*|x_t) \leq 1$ and $P(y^*|x_{NN}) \leq 1$,

$$\begin{aligned}\epsilon_{NN} &= P(y^*|x_t)(1 - P(y^*|x_{NN})) + P(y^*|x_{NN})(1 - P(y^*|x_t)) \\ &\leq 1(1 - P(y^*|x_{NN})) + 1(1 - P(y^*|x_t))\end{aligned}$$

As, $x_{NN} \rightarrow x_t$, $P(y^*|x_{NN}) = P(y^*|x_t)$

$$\epsilon_{NN} \leq (1 - P(y^*|x_t)) + (1 - P(y^*|x_t))$$

$$\epsilon_{NN} \leq 2(1 - P(y^*|x_t))$$

$$\epsilon_{NN} \leq 2\epsilon_{Bayes}$$

As $n \rightarrow \infty$, the 1-NN classifier is only a factor 2 worse than the best possible classifier.

K Nearest Neighbors

The Curse of Dimensionality



Sources

- Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 2, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- The Curse of Dimensionality**, Aaron Lipeles, <https://towardsdatascience.com/the-curse-of-dimensionality-f07c66128fe1>
- Wiki K-Nearest Neighbors**: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- Effects of Distance Measure Choice on KNN Classifier Performance - A Review**, V. B. Surya Prasath et al., <https://arxiv.org/pdf/1708.04321.pdf>
- A Comparative Analysis of Similarity Measures to find Coherent Documents**, Mausumi Goswami et al. <http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf>
- A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data**, Ali Seyed Shirkhorshidi et al., <https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable>
- Cover, Thomas, and, Hart, Peter. Nearest neighbor pattern classification[J]. Information Theory, IEEE Transactions on, 1967, 13(1): 21-27

The Curse of Dimensionality

(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

KNN Assumption

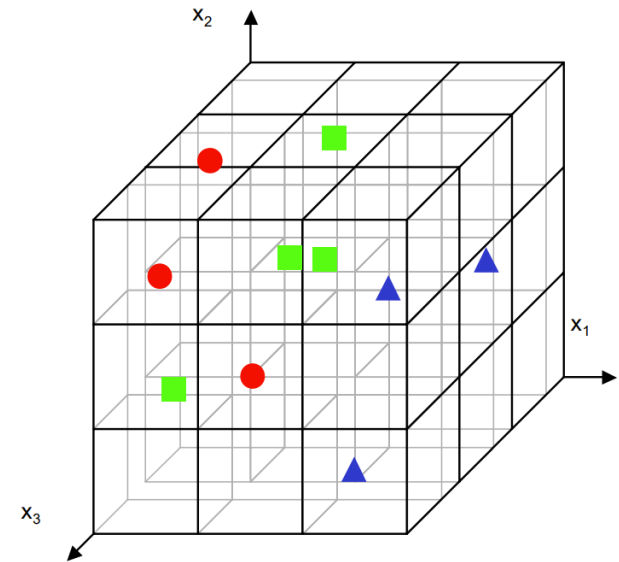
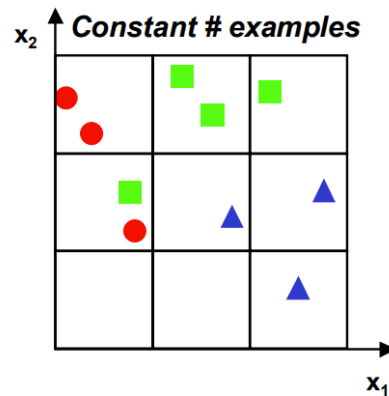
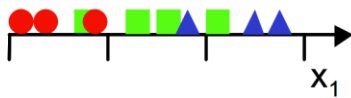
- Points near one another tend to have the same label
- So, KNN tries to find the nearest points

Problem

- In high dimensions, points drawn from a uniform distribution, are never near one another
 - High dimensional spaces are sparsely populated
- So, there are no neighbors near you!
 - You have “nearest” neighbors but no “near” neighbors!
 - This contradicts the basic assumption of KNN (above)

Sparsely Populated Spaces

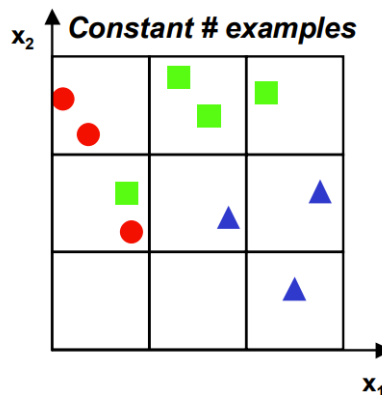
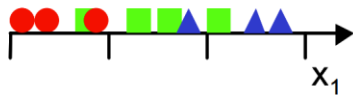
- As dimensionality grows, all/most regions of space get sparsely populated
 - Fewer observations per region
 - E.g., 10 observations spread across:
 - 1d: 3 regions, 2d: $3^2 = 9$ regions, 1000d: 3^{1000} regions



Why is High Dimensionality Bad, in general?

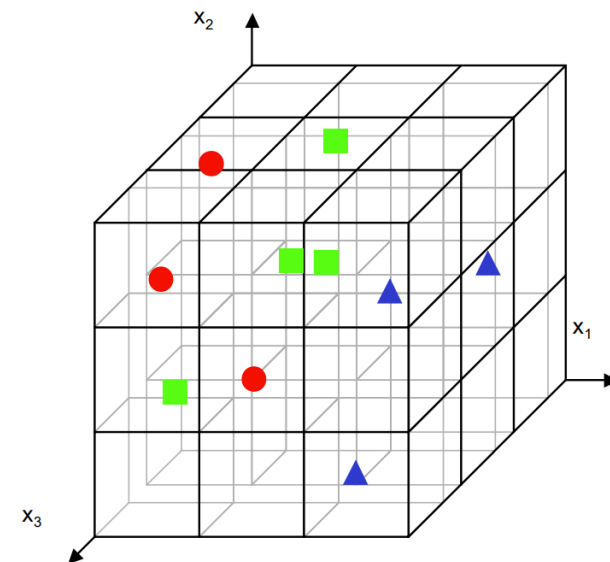
- A small family of instances living in a huge house: What's the problem?
- Machine Learning methods are statistical:
 - Count observations in various regions of some space
 - Use counts to predict outcomes
 - E.g. Naïve Bayes, Decision Trees
- As dimensionality grows, all/most regions of space get sparsely populated
 - Fewer observations per region
 - E.g., 10 observations spread across:

- 1d: 3 regions, 2d: 3^2 regions, 1000d: 3^{1000} regions



- Statistics needs repetitions:

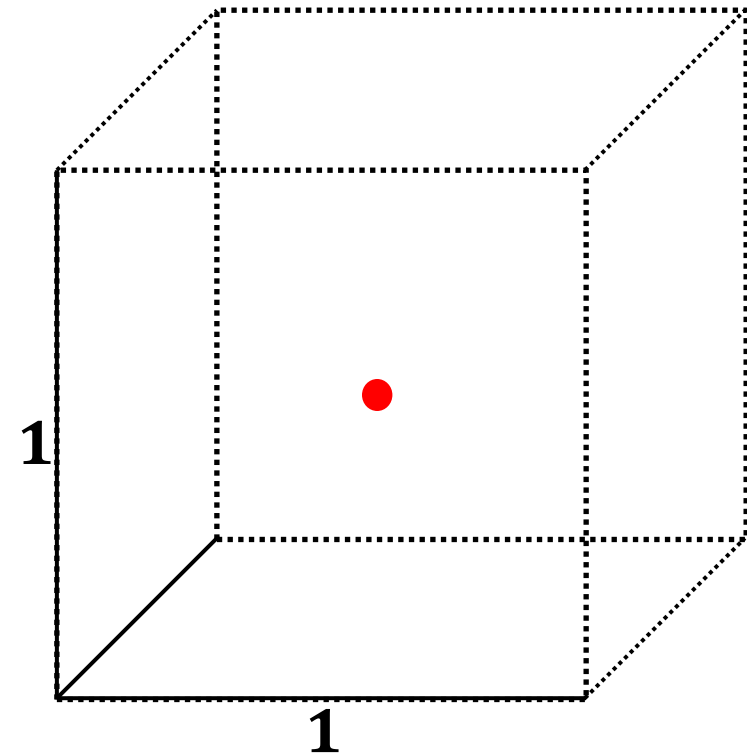
- Flip a coin once $\rightarrow$ head
- $P(\text{head}) = 100\%$?



Demonstration1

(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

- Let's draw points randomly (sampled from a uniform distribution) from the unit cube
- How much space the k nearest neighbors of the red test point will occupy?



Demonstration 1

(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

- Imagine the unit cube $[0,1]^d$
- All training data is sampled uniformly within this cube

$$\forall i, x_i \in [0,1]^d$$

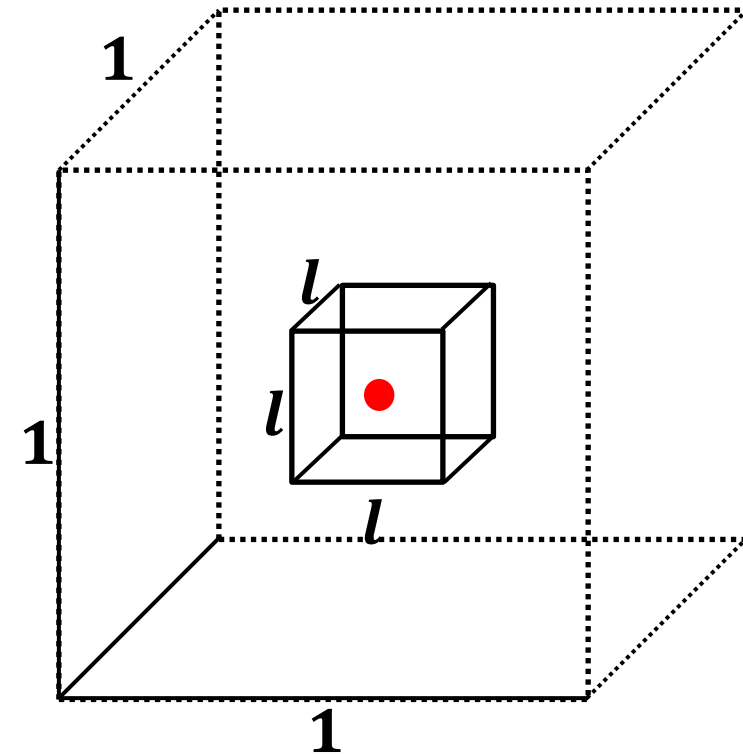
- Now, let l be the edge length of the *smallest* cube that contains the k nearest neighbor of the test point.
- The ratio between volumes of the small cube l^d and unit cube 1^d is:

$$l^d \approx \frac{k}{n}$$

as the points are drawn uniformly. Therefore:

$$l \approx \left(\frac{k}{n}\right)^{\frac{1}{d}}$$

- If $k = 10$ and $n = 1000$. How big is l ?

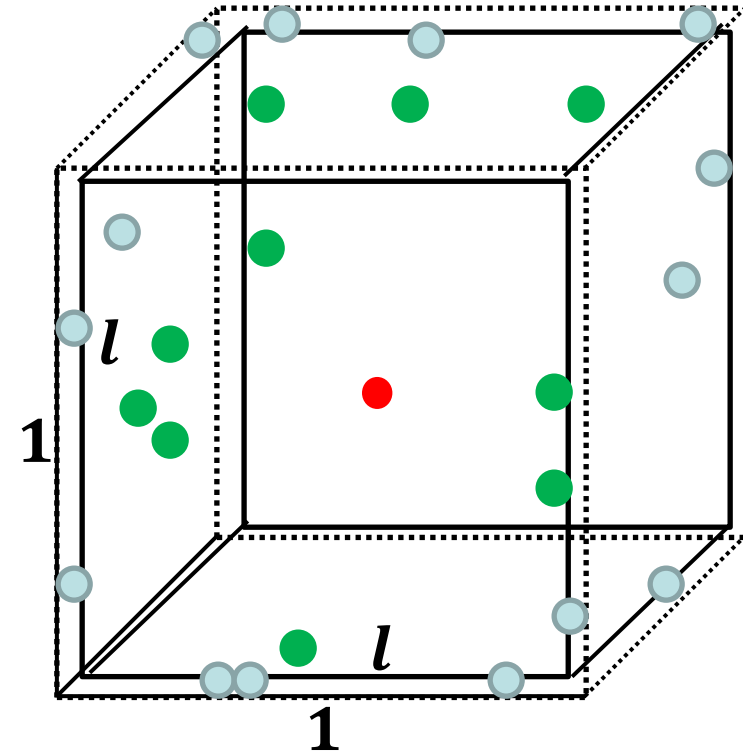


d	l
2	0.1
10	0.63
100	0.955
1000	0.9954

Demonstration 1

(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

- If $k = 10$ and $n = 1000$. How big is l ?
- So, the test point has no neighbors near it!
- Most “nearest” neighbors (green) are close to the edges, otherwise we would have drawn a smaller cube around them
- So why should the test point share its label with these faraway points? The whole premise fails!



d	l
2	0.1
10	0.63
100	0.955
1000	0.9954

Demonstration 1

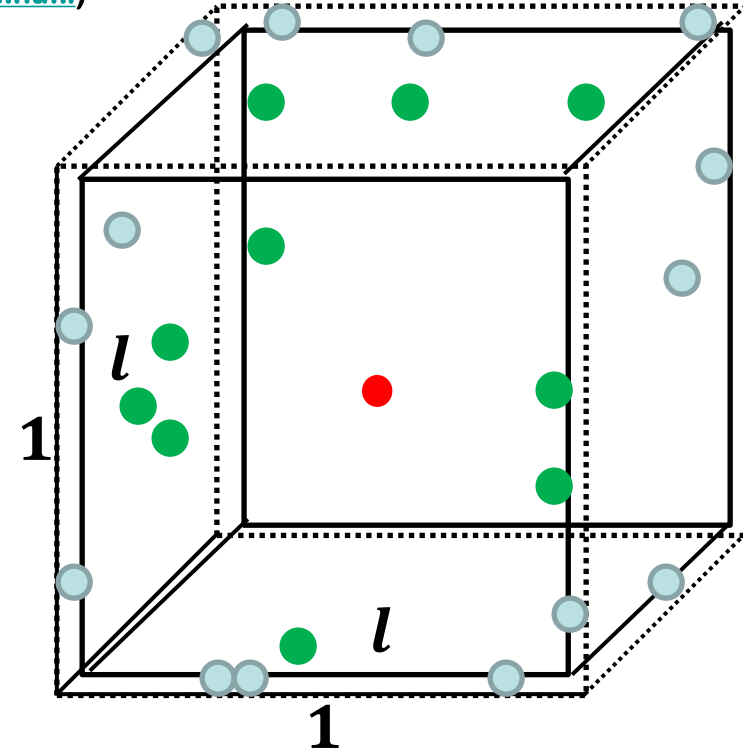
(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

- How about we increase the number of training samples, n , until the nearest neighbors are truly close to the test point?
- How many data points do we need such that l becomes truly small?

- If $l = \frac{1}{10} = 0.1$
 - i.e., we want the k nearest neighbors to fit in one tenth of the space of the unit cube

$$l^d \approx \frac{k}{n} \Rightarrow \quad n = \frac{k}{l^d} = k \cdot 10^d$$

- This grows exponentially!
- For $d > 100$ we would need far more data points than there are electrons in the universe!
 - The current estimate is 10^{81} in the observable universe.



d	l
2	0.1
10	0.63
100	0.955
1000	0.9954

Another example https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html

- Assume a unit line segment
- ϵ is the length of very small segments on both sides of it
- What is the probability of drawing a point uniform randomly from the segment such that it does not lie on the edges ϵ ?

$$P(N.E.point) = \left(\frac{1 - 2\epsilon}{\text{length of the segment}} \right) \\ = 1 - 2\epsilon$$

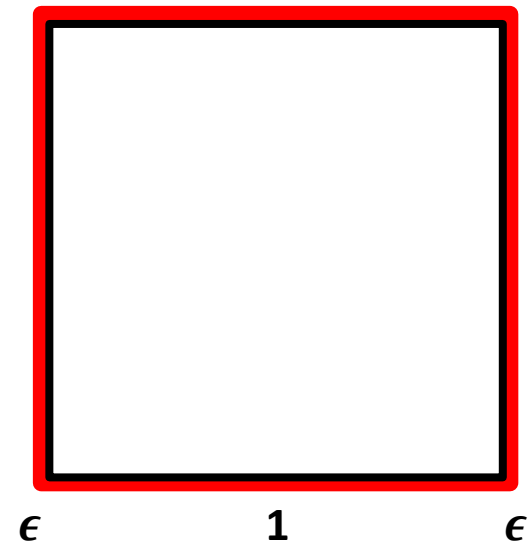
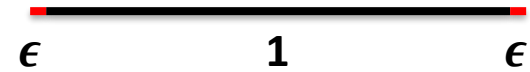
- What if we convert the line segment into a unit square. What is the probability of N.E. point now?

$$P(N.E.Point) = (1 - 2\epsilon)^2$$

- How about a d -dimensional cube?

$$P(N.E.Point) = (1 - 2\epsilon)^d$$

- As, $1 - 2\epsilon < 1$, this will reach 0 very rapidly!
- A point will become an edge point even if it is an edge point in any one dimension. So, the probability of getting a non-edge point in every single dimension is near to 0.



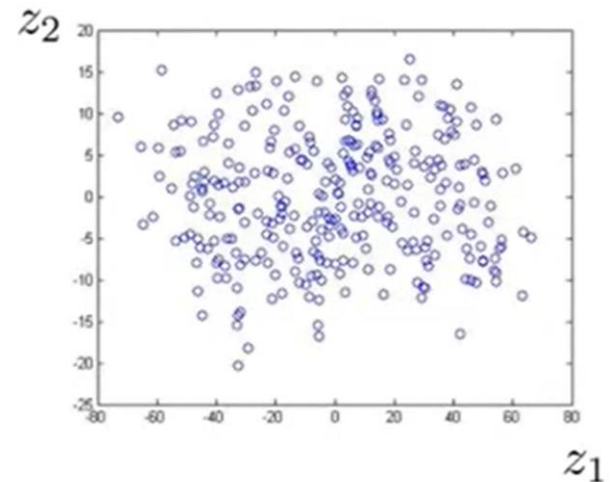
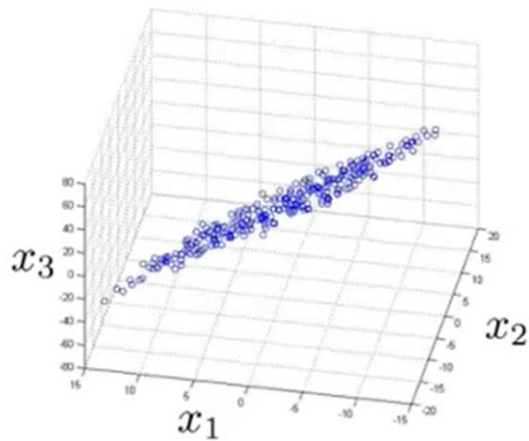
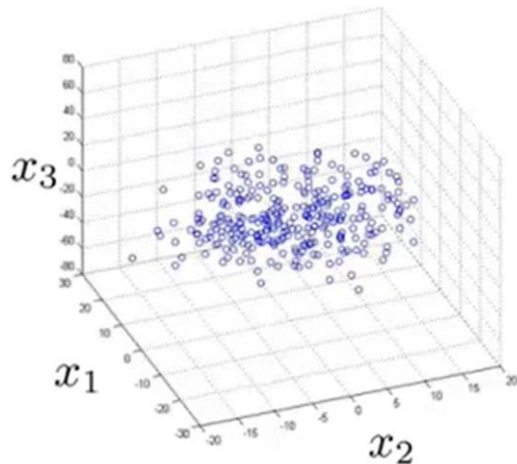
So how does KNN work at all?

(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

- The basic assumption of the Curse of Dimensionality is that the instances are spread uniformly across the n -dimensional space
- Many practical problems (SPAM vs. Not SPAM with bag-of-words, face recognition) etc. have **high ambient dimensionality** (SPAM vs. Not SPAM has $|V|$, face recognition may have [number of pixels] e.g., 18M) but **low intrinsic dimensionality**
 - May be a small subset of $|V|$ helps with the SPAM vs. Not SPAM classification
 - May be the actual face recognition features are much less e.g., humans recognize face based on m/f, color, hair color, spectacles, shape of the nose etc.
- So, the true or effective dimensionality might be much lower making KNN still feasible.

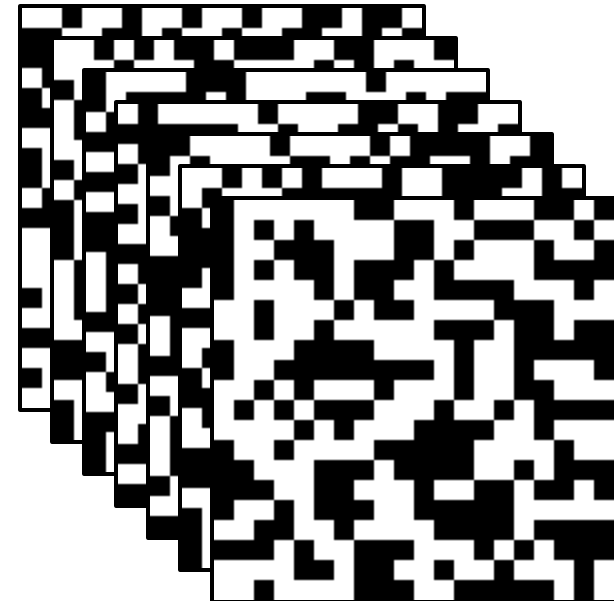
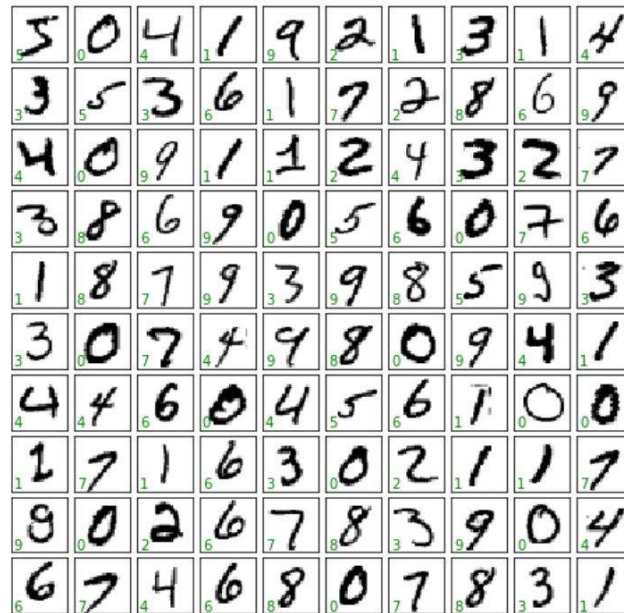
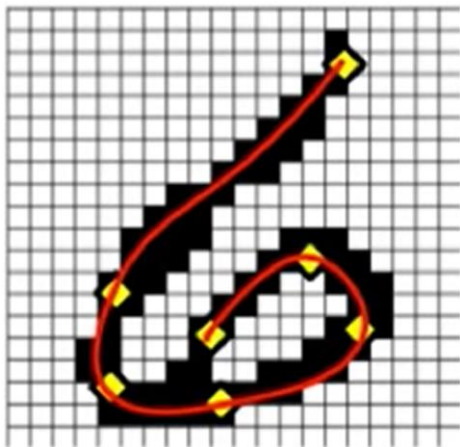
Dimensionality: Real vs. Apparent

- ML datasets are typically high dimensional
 - Bag-of-word features
 - Images: As each pixel is an attribute, we have 2×10^6 features for a 2megapixel image
 - Text: As each word type is a feature, we can easily get a million features
 - The high dimensionality is apparent – not real!
 - It is an artifact of the way we store attributes
 - The real dimensionality is often much lower
 - A surface or manifold (sheet) embedded in a high ambient dimensional space



The Curse of Dimensionality

- High dimensional spaces are sparsely populated
- Hand-written digits
 - 28 x 28 bitmaps: 784 features and $\{0,1\}^{784}$ possible events (feature values)
 - However, most of these feature value combinations will never be observed
 - Actual digits only occupy a tiny fraction of this 784-dimensional space. Digits are very rare in that space!
 - True dimensionality: Possible variations of pen strokes constrained by the laws of physics
 - Try drawing random events from this space!



The Curse of Dimensionality

Another example: Words

- Example paragraph, 62 tokens, 46 word types, *Through the Looking-glass* by Lewis Carroll

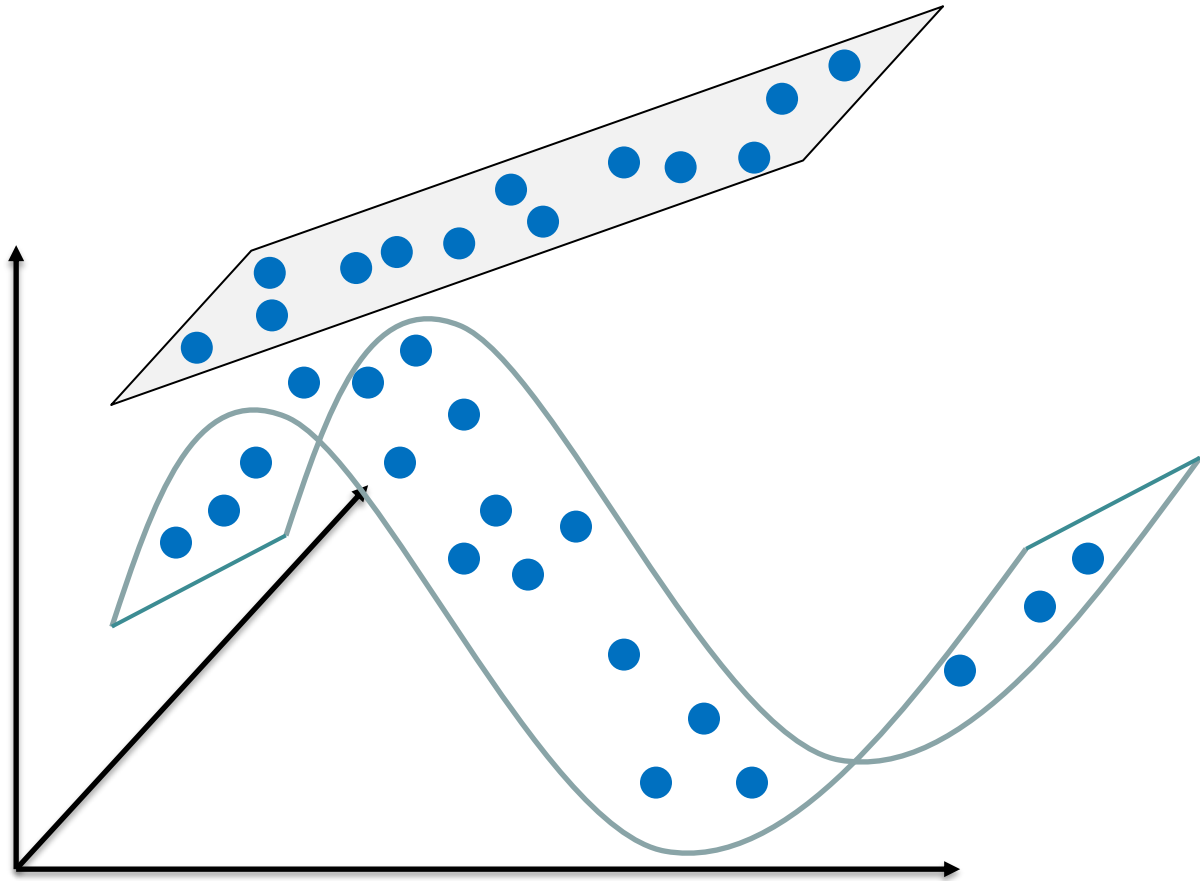
One thing was certain, that the white kitten had had nothing to do with it. It was the black kitten's fault entirely. For the white kitten had been having its face washed by the old cat for the last quarter of an hour and bearing it pretty well, considering; so you see that it couldn't have had any hand in the mischief.

- Using this vocabulary, we generate random sequences of length 10 (46^{10} potential sequences)
- Most of the random combinations of these features make no sense
 - for fault kitten's one entirely to last considering and that
 - having having having for its any so old one mischief
 - of couldn't do to so it considering well bearing mischief
 - last with entirely been of you entirely face with white
 - entirely it entirely so white well an one white face
 - you having old in face entirely by thing hour hour
 - white one see considering quarter do so bearing been do
- **Most of the potential strings would be meaningless!**
 - **So, the true dimensionality must be lower.**

So how does KNN work at all?

(https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html)

- The data may lie on a lower-dimensional subspace or manifold where Euclidean distances may hold locally



K Nearest Neighbors Enhancements

Sources

- Nearest Neighbor Methods**, Victor Lavrenko, Assistant Professor at the University of Edinburgh,
https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ
- Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 2,
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- Wiki K-Nearest Neighbors**: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- Effects of Distance Measure Choice on KNN Classifier Performance - A Review**, V. B. Surya Prasath et al., <https://arxiv.org/pdf/1708.04321.pdf>
- A Comparative Analysis of Similarity Measures to find Coherent Documents**, Mausumi Goswami et al. <http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf>
- A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data**, Ali Seyed Shirkhorshidi et al.,
<https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable>
- Cover, Thomas, and, Hart, Peter. **Nearest neighbor pattern classification**. Information Theory, IEEE Transactions on, 1967, 13(1): 21-27

Parzen Windows

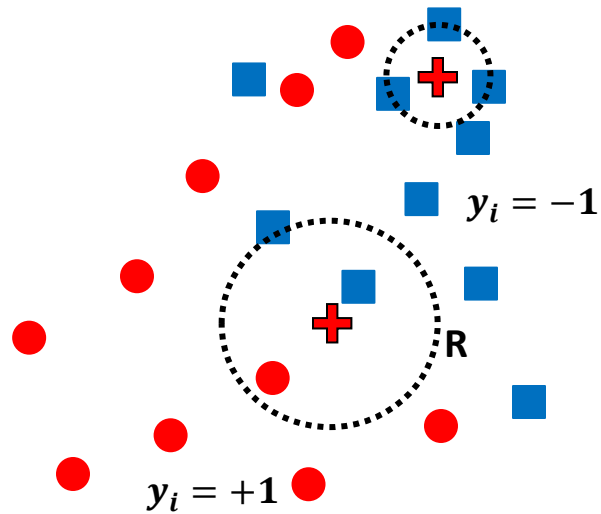
- **Parzen Windows** or **kernel density estimation (KDE)**, is a non-parametric approach to estimate the probability density function (pdf) of a random variable.
- Particularly used for pattern classification
- Useful for tackling problems where parametric estimations are not suitable because the underlying distribution is unknown or difficult to identify.

Parzen Windows

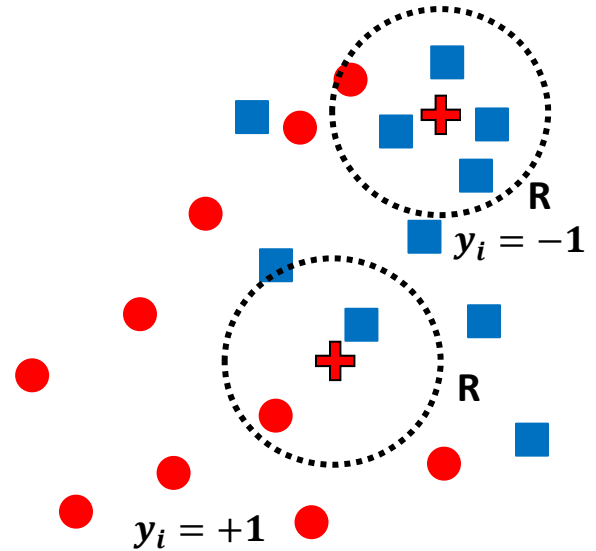
- **Idea:** Given a set of data points, Parzen Windows estimate the pdf at any point in the space by counting the number of data points in a window centered around that point.
- **Window Function:** A window function (or kernel function) K and a window width h are used to define the window.
 - Common kernel functions include Gaussian, rectangular, and Epanechnikov kernels.

Parzen Windows and Kernels

3-NN



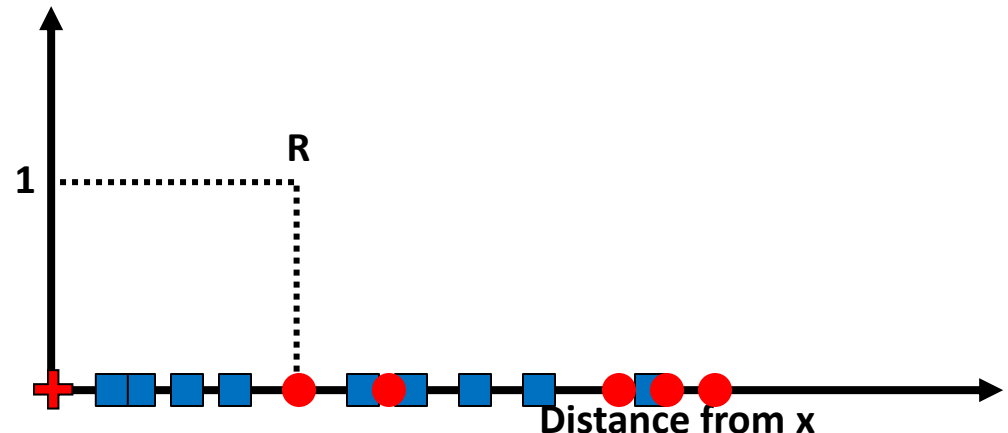
Parzen Window



Parzen Windows

$$f(x) = \text{sgn} \left[\sum_{i: x_i \in R(x)} y_i \right]$$

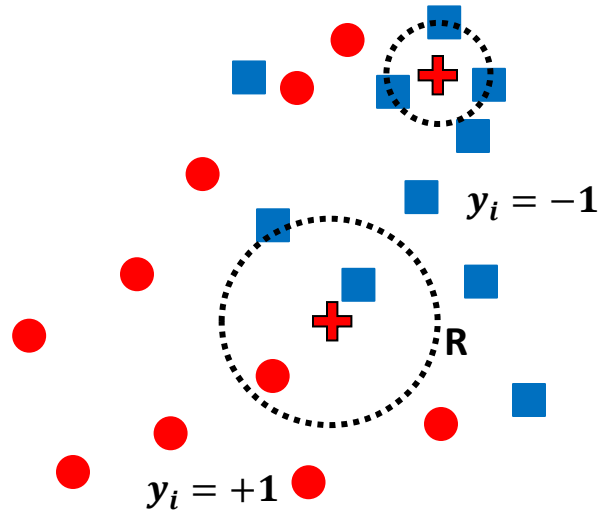
$$f(x) = \text{sgn} \left[\sum_i y_i \cdot 1_{\|x_i - x\| \leq R} \right]$$



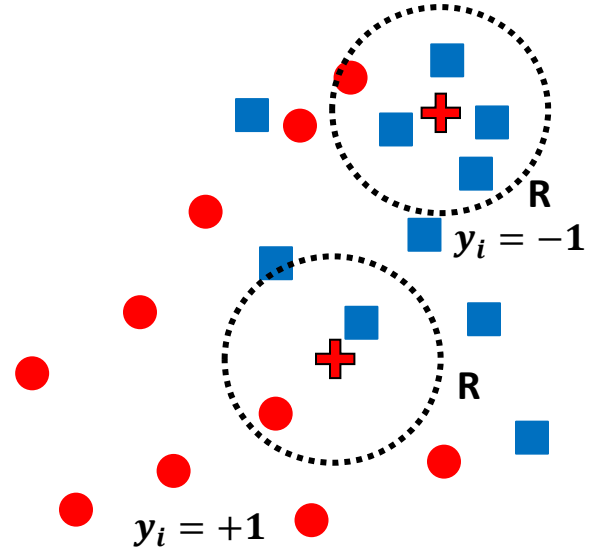
Ref: Victor Lavrenko, University of Edinburgh

Parzen Windows and Kernels

3-NN



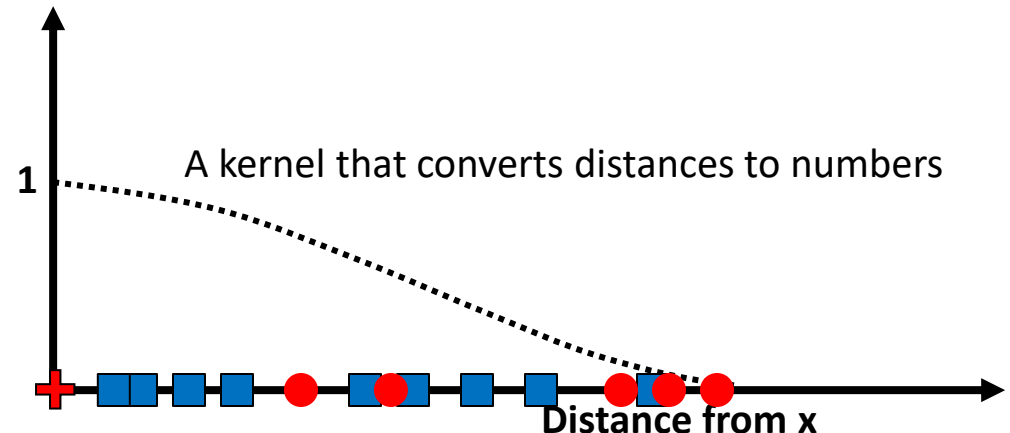
Parzen Window



Parzen Windows

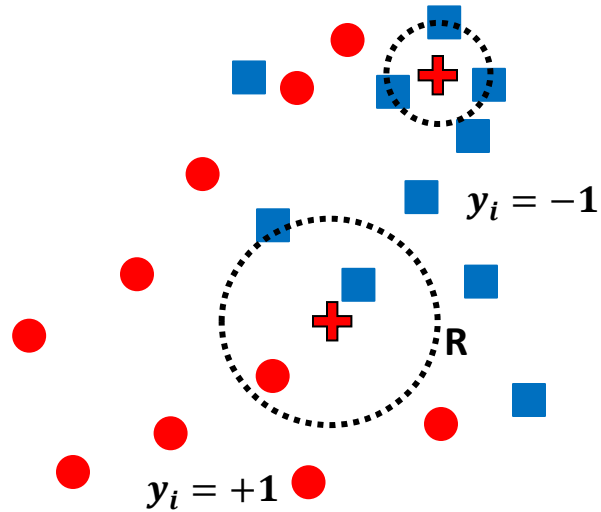
$$f(x) = \text{sgn} \left[\sum_{i: x_i \in R(x)} y_i \right]$$

$$f(x) = \text{sgn} \left[\sum_i y_i \cdot 1_{\|x_i - x\| \leq R} \right]$$

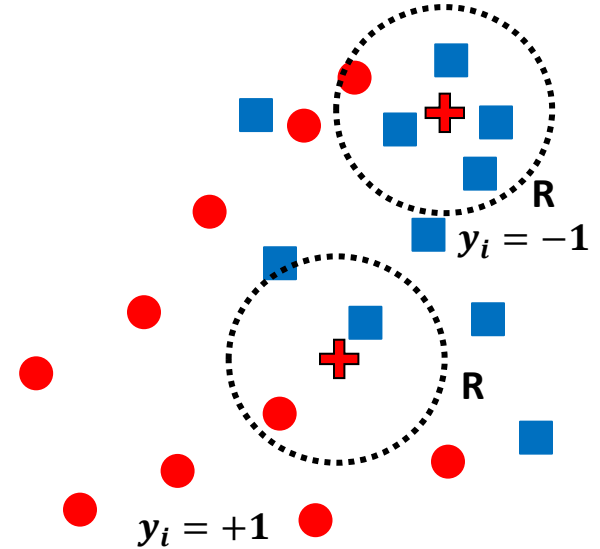


Parzen Windows and Kernels

3-NN



Parzen Window



Parzen Windows

$$f(x) = \text{sgn} \left[\sum_{i: x_i \in R(x)} y_i \right]$$

$$f(x) = \text{sgn} \left[\sum_i y_i \cdot 1_{\|x_i - x\| \leq R} \right]$$

Kernelized Parzen window

$$f(x) = \text{sgn} \left[\sum_i y_i K(x_i, x) \right]$$

vs

$$f(x) = \text{sgn} \left[\sum_i \alpha_i y_i K(x_i, x) \right]$$

Performance of KNN Algorithm

Time complexity: $O(nd)$

Dimensionality Reduction:

- Use dimensionality reduction techniques like Principal Component Analysis (PCA) or Linear Discriminant Analysis (LDA) to reduce the number of features, which can significantly speed up distance computation.

Instance Reduction:

- Reduce the number of training instances by removing redundant or noisy samples. Techniques such as condensed nearest neighbor (CNN) can be used for instance reduction.
 - An under-sampling technique that seeks a subset of a collection of samples that results in no loss in model performance, referred to as a minimal consistent set.
 - Enumerate the examples in the dataset and add them to the “store” only if they cannot be classified correctly by the current contents of the store.

Performance of KNN Algorithm

Time complexity: $O(nd)$

Using More Efficient Distance Metrics:

- Choose distance metrics that are less computationally intensive. For instance, Manhattan distance may be computed faster than Euclidean distance in high-dimensional spaces.

Using Approximate Nearest Neighbor Algorithms:

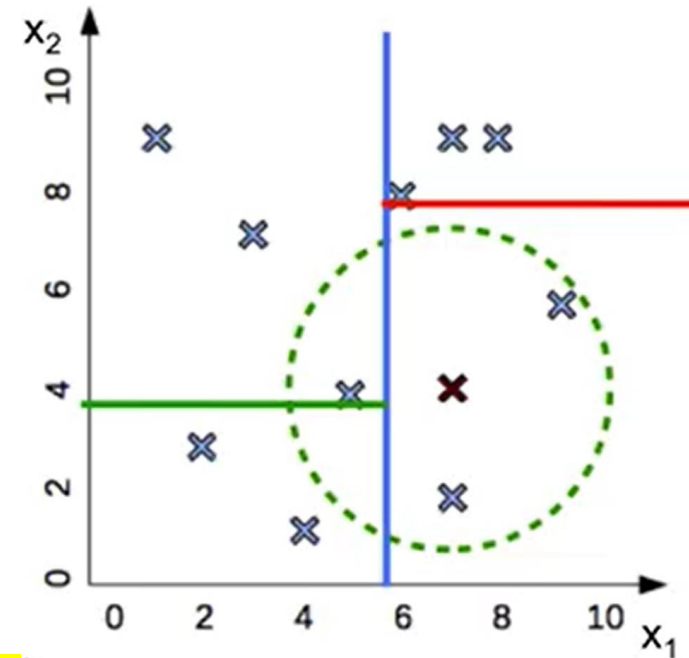
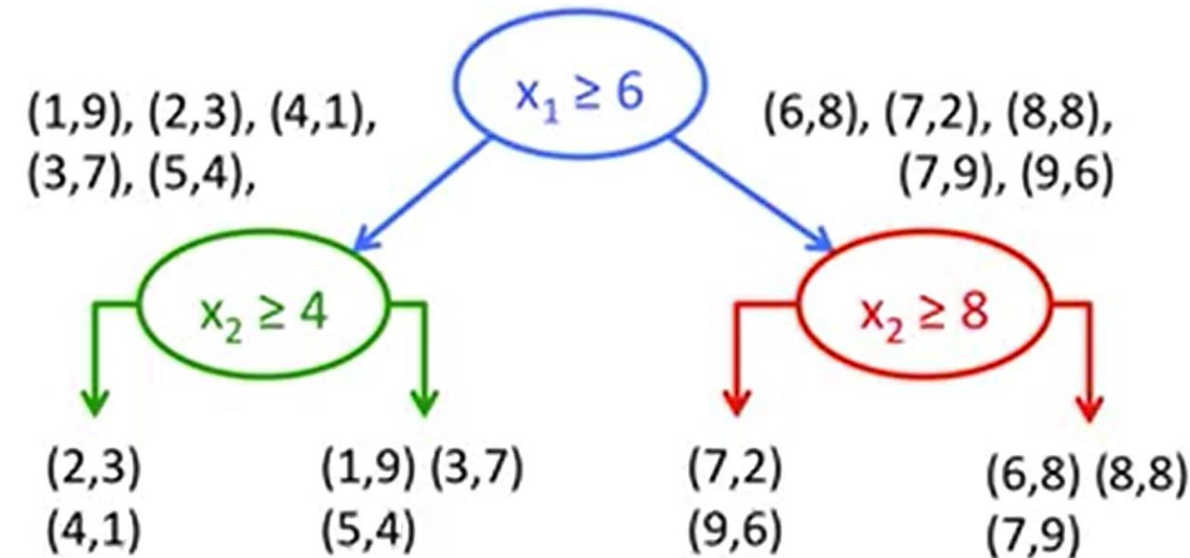
- Algorithms like Locality-Sensitive Hashing (LSH) or k-d trees can be used to approximate the nearest neighbors, significantly reducing computation time.

Performance of KNN Algorithm

- Time complexity: $O(nd)$
- Reduce d : Dimensionality reduction
- Reduce n : Compare to a subset of examples
 - Identify $m \ll n$ potential near neighbors to compare against
 - $O(md)$
 - **K-D trees**: Low-dimensional, real-valued data
 - $O(d \log_2 n)$, only works when $d \ll n$, **inexact**: can miss neighbors
 - **Locality-sensitive hashing**: high-dimensional, real or discrete
 - $O(n'd)$, $n' \ll n$, **inexact**: can miss neighbors
 - **Inverted lists**: High-dimensional, discrete (sparse) data
 - $O(n'd')$, where $d' \ll d$, $n' \ll n$, only for sparse data (e.g. text), **exact**

K-D Trees

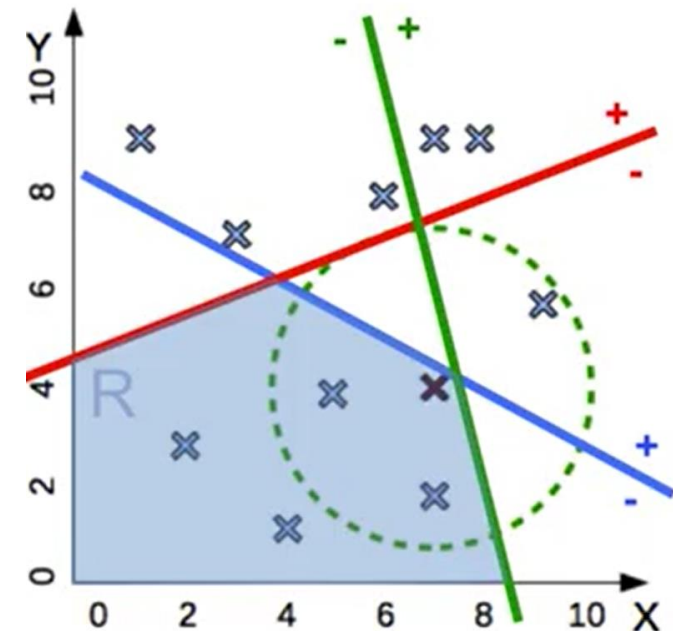
- Pick a random dimension, find median, split data, repeat
(1,9), (2,3), (4,1), (3,7), (5,4), (6,8), (7,2), (8,8), (7,9), (9,6)
- $O(d \log_2 n)$
- E.g. test point: (7,4)
 - Compare with all the points in the region
- Can easily miss nearest neighbors



Example ref: Victor Lavrenko, University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ

Locality-sensitive Hashing

- Draw random hyper-planes $h_1, \dots, h_k$
- The space is sliced into 2^k regions
 - Polytopes: Flat sides, bounded by hyperplanes
 - Mutually exclusive
- Compare x only to training points in that region
- Complexity: $O(d \log n)$ if $k \approx \log n$
- Inexact: Can miss neighbors
 - Repeat with different hyperplanes
- Why do we need these?
 - In case of K-D trees, in high dimensions, someone could be your neighbor in $d-1$ dimensions, but still very far away in the d th dimension
 - LSH cuts across dimensions rather than doing it one by one



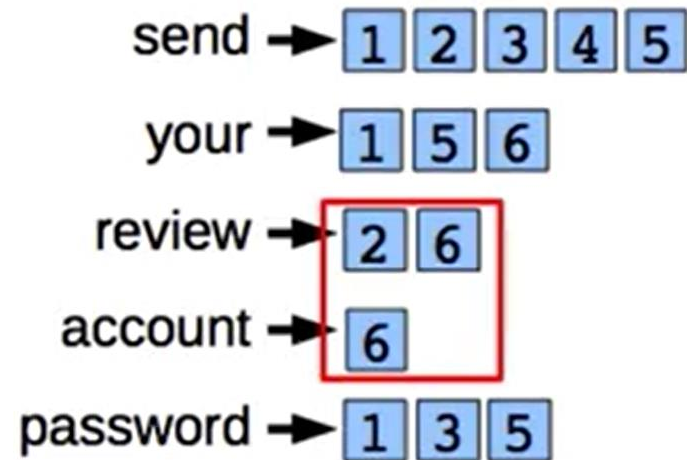
Example ref: Victor Lavrenko, University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ

Inverted Lists

- High dimensional, sparse data
- New email: “account review”
- $O(d\sqrt{n})$, where d : non-zero attributes, $\sqrt{n}$: avg length of list
- Exact: does not miss neighbors

D1: “send your password”	spam
D2: “send us review”	ham
D3: “send us password”	spam
D4: “send us details”	ham
D5: “send your password”	spam
D6: “review your account”	ham

new email: “account review”



Example ref: Victor Lavrenko, University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ

Conclusion

The k-Nearest Neighbors (k-NN) algorithm is a simple, instance-based learning algorithm.

Strengths:

- 1. Simple and Intuitive:** k-NN is easy to understand and implement.
- 2. No Training Phase:** k-NN doesn't require a training phase. New data can be added seamlessly.
- 3. Non-Parametric:** It makes no assumptions about the underlying data distribution.
- 4. Multiclass Classification:** Can be used for multiclass classification problems.
- 5. Versatile:** Can be used for classification, regression, and search tasks.

Conclusion

Weaknesses:

1. **Computational Complexity:** Computationally expensive, especially with large datasets as it requires computing the distance to all training samples for each query.
2. **Memory Intensive:** Requires storing the entire dataset, which can be memory-intensive for large datasets.
3. **Sensitive to Irrelevant Features:** The performance of k-NN can degrade with irrelevant or redundant features because it uses all features equally in the distance calculation.
4. **Sensitive to the Scale of the Data:** Different scales of features can impact the performance of k-NN. Feature scaling is often required.
5. **Optimal k Value:** Choosing the optimal number of neighbors (k) can be challenging.

For more details please visit
<http://aghaaliraza.com>



Thank you!